

ANS Forth Test Suite

Introduction

The ANS Forth Test Suite is intended to prove that a forth distribution is in compliance with the ANS Forth standard.

However full compliance requires other conditions to be satisfied.

Source: <https://forth-standard.org/standard/testsuite> (Annex F), fetched 2026-07-15. Explicitly redistributable per the test harness's own header:
"(C) 1995 JOHNS HOPKINS UNIVERSITY / APPLIED PHYSICS LABORATORY - MAY BE DISTRIBUTED FREELY AS LONG AS THIS COPYRIGHT NOTICE REMAINS."

The test suite files here have been modified to facilitate development testing, instead of compliance testing. This includes development by manual cut/paste of forth source into a terminal emulator or by running the supplied automation script.

Modifications include:

1. File splits by section.
2. Sections can be executed individually, without dependencies on prior tests.
3. Performance enhancements.
4. Improved reporting (`TEST-BEGIN` / `TEST-END` , see *Test Reporting* below).
5. Corrections for test expectations that violate the standard.
6. Adaptations for a 16-bit cell.

Testing can target serial-connected hardware (MECB 6809) or the MAME software emulation of a 6809 SBC with a 6850 ACIA.

Running the Suite

MAME Emulator Setup

The MAME emulator needs to be setup in advance. Refer to the instructions in

```
${HOME}/git/ClaudeForth/MECB6809\ Emulation.
```

This includes, from one terminal:

1. Replacing the buggy 6850acia.cpp file provided in the MAME git project with the modified version provided in the ClaudeForth git project.
2. Installing the mecb6809.cpp driver source file.
3. Building the MAME project.
4. Assembling then installing, the forth6809.asm as a .bin file into the MAME rom directory.

```
# The update to the MAME executable.
cp \
    ${HOME}/git/ClaudeForth/MECB6809\ Emulation/6850acia.cpp \
    ${HOME}/git/mame0288/src/devices/machine/6850acia.cpp
cp \
    ${HOME}/git/ClaudeForth/MECB6809\ Emulation/mecb6809.cpp \
    ${HOME}/git/mame0288/src/mame/homebrew/mecb6809.cpp
cd ${HOME}/git/mame0288
make SUBTARGET=mecb6809 SOURCES=src/mame/homebrew/mecb6809.cpp TOOLS=1 REGENIE=1
```

Then in another terminal:

```
# Providing the ROM file.
cd ${HOME}/git/ClaudeForth
lwasm --6809 --format=raw \
    --output=forth6809.bin --list=forth6809.lst \
    --define=UNITTESTS=0 --define=TSTSELECTOR=15 \
    --define=SERIALPOLL=1 \
    forth6809.asm
mkdir "${HOME}/Library/Application Support/mame/roms/mecb6809"
cp forth6809.bin "${HOME}/Library/Application Support/mame/roms/mecb6809/mecb6809"
```

Python Test Runner

Usage

In a further terminal,
assuming a git clone to download this project,
navigate to the top level ClaudeForth directory
and then to the ANS Test Suite. Execute the test runner script:

```
# Go there
```

```

cd ${HOME}/git/ClaudeForth/ANS\ Test\ Suite
# Verify
ls -al
# Execute the tests suite
python3 ans_test_runner.py \
    --target mame \
    --mame-bin "$HOME/git/mame0288/mecb6809" \
    --retrigger-cmd $HOME/git/ClaudeForth/ANS\ Test\ Suite/retrigger.cmd \
    --skip-assemble \
    --ans-dir $HOME/git/ClaudeForth/ANS\ Test\ Suite/ans_tests \
    --sections 08 09 10 11 12 13 14 15 16 17 18 19 20 21 23 24 26 \
    --char-delay 0.05 \
    --retries 0 \
    --log-dir $HOME/git/ClaudeForth/ANS\ Test\ Suite/ans_test_results

```

`ans_test_runner.py` automates loading the setup forth files into a running forth6809 image.

Then it loads the forth tests.

The test runner reports a summary PASS/FAIL

per section into the stderr of the terminal.

The full transcript of each section,

including the `TEST SUMMARY` and any `ttester` failure messages,

is written to a per-section log file in the `--log-dir` directory.

A section `PASS`es when its `TEST SUMMARY` line reports zero failures

and the section loaded without a compile error or line timeout.

A section with no `TEST SUMMARY` line at all `FAIL`s.

The runner prints a warning if it sees stack imbalance notes.

To locate the data stack cell at which a section leaks,

add the `--trace-depth` option.

The runner then asks the target for `DEPTH` after every line it sends.

Operating Principles

The python test runner talks to either a MAME instance

(spawned the same way `run_all_tests.sh` does,

over a `-rs232 null_modem -bitb socket.host:port` TCP connection)

or a real serial-connected MECB6809 with the Forth

binary in EPROM (via pyserial).

The test runner applies per character delays, but doesn't apply end-of-line delays.

Instead it uses forth6809's *own* line-by-line `ok / ERROR` reply

as the pacing signal.

In other words: one line goes out,
the runner blocks until that line's own reply comes back,
then the next line goes out.

Long test-file lines (some exceed 1000 characters) are automatically
re-wrapped to fit under `TIBBUFL` (80 bytes)
without ever splitting inside a `s" / ." / \` construct
or inside a `T{ ... }T` assertion.

Software (XON/XOFF) flow control from the target is honoured.

The runner also removes the dead `HAS-FLOATING` branches of `ttester.fs`
before sending it (the target has no floating-point word set),
which removes most of the harness load time.

See the script's own `--help` and
module docstring for every option.

Manual Cut and Paste Via Terminal Emulator

Usage

For full communication details refer to the file:

```
more ${HOME}/git/ClaudeForth/MAME_to_minicom.md
```

In yet another terminal start up the socat bridge:

```
socat PTY,link=$HOME/mame-pty,raw,echo=0,ixon=1,ixoff=1 TCP-LISTEN:11185,reuseadd
```

This must be started first.

In yet another terminal startup minicom:

```
minicom 57600_XONXOFF
```

In yet another terminal startup MAME as the MECB 6809 emulator:

```
cd $HOME/git/mame0288  
./mecb6809 mecb6809 \  
-rs232 null_modem -bitb socket.127.0.0.1:11185 \  
-throttle \  
-window -resolution 640x480 \  

```

-debug

The terminal emulator should now respond to keyboard input, with the per character delay and handshaking preset.

In a text editor supporting copy and paste, choose a forth source code file, select the text, copy it to the clipboard and then paste it into the minicom window. All the files are loaded in this way, in the same order as used by the python test runner:

1. 00a_tool_ext_conditionals.fs
2. ttester.fs
3. 00_test_prelude.fs
4. any section files, in any order.

Each section file prints its own `TEST SUMMARY` at its end, so no runner is needed to read the result.

Do not paste a file that has been reflowed by an editor: see *Editing Test Files* below.

Test Reporting

Each section file reports for itself, using two words defined in `00_test_prelude.fs` :

Word	Purpose
TEST-BEGIN	Zeroes the pass and fail counters (<code>TESTCOUNT</code> , <code>FAILCOUNT</code> , incremented by <code>T</code> in <code>ttester.fs</code>) and records the current data stack depth.
TEST-END	Prints the file's statistics, then checks the data stack depth against the depth recorded by <code>TEST-BEGIN</code> . Always prints in DECIMAL and preserves <code>BASE</code> .

Every section file calls `TEST-BEGIN` on the line after its `MARKER Mnn` and `TEST-END` on the line before its closing `Mnn` .

The prelude calls them around its own checks as well.

Typical output:

```
TEST SUMMARY: 125 run, 0 failed
```

If a file leaves the data stack deeper than it found it:

```
TEST SUMMARY: 31 run, 0 failed
STACK IMBALANCE: 3 extra cell(s) dropped
```

The extra cells are dropped so the next file starts clean.

A shallower stack is reported as `n cell(s) missing`.

The stack check matters because `T{ ... }T` measures depth relative to its own starting depth, so a stray cell left by an earlier test is invisible to it.

A file can show zero failures and still leak cells.

When a test fails, `ttester` prints a message such as `INCORRECT RESULT:` followed by the test source line.

A failure counted in `TEST SUMMARY` always has such a message earlier in the output.

Editing Test Files

A few tests depend on the physical line layout of the file, because the target's `ACCEPT` gives each test exactly one input line.

These must stay split exactly as they are:

- `19_string_words.tests.fs`: `T{ BL GS3 / DROP -> 0 }T`, and both `T{ PARSE-NAME / NIP -> 0 }T` pairs. They need an empty parse area, i.e. the end of the line must be reached.
- `24_environmental_query.tests.fs`: the `>IN` tests, the three-line `RESCAN?` test, and `GS4`.

Other guidance:

- Keep one test per line, so a failure message identifies it.
- Never put a test on the same physical line as a `\` comment. The rest of the line is swallowed and the test silently never runs.
- A bare `DECIMAL` or `HEX` leaks into the next section file, because `MARKER` restores the dictionary but not `BASE`. Restore it unconditionally, as the existing files do.
- `>R / R>` must not span separate top-level lines.
- Keep `TEST-BEGIN` after `MARKER Mnn` and `TEST-END` before the closing `Mnn`. The tool script `apply_test_markers.py` inserts both idempotently.

Manifest

1. Python test runner to automate testing.

2. Support files used by the python test runner.
3. Forth setup files that are loaded into the test target prior to running tests.
These are located in the sub-directory `ans_tests`.
4. The `ans_tests` sub-directory containing the forth test files
organised according to the glossary sections for forth6809.
5. The `ans_test_results` sub-directory containing the log files
recording previous test run results.
6. Miscellaneous utility scripts.

What follows is a more detailed description of the contents of `ans_tests` and the function of each file.

Forth Conditional Compilation Support

The utility file `00a_tool_ext_conditionals.fs` is loaded first to provide conditional forth compilation for floating point testing support when the following `ttester.fs` file is compiled.

This file provides `[IF] / [ELSE] / [THEN]` from the Programming-Tools word set.
These words are not provided by ClaudeForth.

`ttester.fs` needs them immediately, in its own conditional compilation of the `HAS-FLOATING / HAS-FLOATING-STACK` source blocks, so this file loads first, ahead of even `ttester.fs`.

`[IF] / [ELSE] / [THEN]` is provided as a reference implementation by the standard itself (forth-standard.org/standard/tools/BracketELSE), built only from words `forth6809.asm` already has (`BL WORD COUNT S" COMPARE REFILL IF/ELSE/THEN BEGIN/WHILE/REPEAT/UNTIL ?DUP EXIT IMMEDIATE`) — nothing further needs to be assembled into the ROM to run the ANS suite.

ANS Forth Test Harness

`ttester.fs` — represents the test harness itself
(ANS Forth Standard, Annex F, Section F.2.3).

It defines the test setup/assertion words `T{ / -> / }T` and every supporting word, including:

1. `ERROR`
2. `EMPTY-STACK`

3. `HAS-FLOATING / HAS-FLOATING-STACK` detection.
4. `X}T / R}T` assertion words for mixed cell/float stack pictures.

Load this file first, before the prelude or any section file.

The extraction of this file was validated by checking nesting of:

1. `[IF]` and `[THEN]` . Counts match exactly (8/8)
2. `:` / `;` counts match exactly (63/63).

No unbalanced conditional-compilation or colon definitions were introduced while extracting it from the original source page.

The original `TEST-REPORT` word has been removed from this file.

Reporting is now done by `TEST-BEGIN / TEST-END` in the prelude.

ANS Forth Test Prelude

The prelude file `00_test_prelude.fs` is loaded after `ttester.fs` , but before the test section files.

It provides shared section setup utilities with support for:

1. Test reporting (`TEST-BEGIN` , `TEST-END`).
2. Two's-complement assumptions (`BITSET?`).
3. Bound constants (`MAX-UINT`, `MAX-INT`, `MIN-INT`, `MID-UINT`, `MID-UINT+1`, `MSB`).
4. Boolean constants (`0S/1S` via `<FALSE>/<TRUE>`).
5. Floored-vs-symmetric division helpers (`IFFLOORED/IFSYM`).
6. String-compare helper (`S=`).
7. Shared memory buffers used by the `FILL/MOVE` tests (`FBUF/SBUF/SEEBUF`).

ANS Forth Tests

The ANS Forth test files, `NN_name.tests.fs` , are aligned with one file per `forth6809.asm` section (matching the exact same numbering and filenames as `forth6809_split/`).

These contain every ANS test block for the words that each section implements.

There are 153 test blocks placed across 17 section files.

Sections that implement no words with an official ANS test have no corresponding file.

For example: section 5's inner-interpreter primitives like `LIT/BRANCH`, have no directly-callable ANS name, so there are no tests.

Everything else in Core (F.6.1) and the Core Extension subset this

system implements (F.6.2) maps to exactly one section.

ANS Forth Test Change Notes

Test File Section Independence

The original ANS suite is one continuous stream where later tests reuse words and constants defined by earlier ones (explicit in the standard's own F.3 narrative: "these are included in the appropriate test").

Splitting it by forth6809.asm section preserved the *word groupings* but broke that original ordering — this has since been fixed so every section file is independently runnable, in any order, any number of times, from a single shared harness+prelude load:

- 09_outer_interpreter.tests.fs (FIND) used to depend on GT1 / GT2 from 13_compiling_words.tests.fs . It now carries its own copy of both (: GT1 123 ; : GT2 ['] GT1 ; IMMEDIATE), so section 13 no longer needs to run first.
- Every section file now opens with `MARKER Mnn` (nn = the section number, e.g. M08) right after its header comment, and closes with a bare invocation of `Mnn` as its last line. `MARKER snapshots` `DPHERE / CODEHERE / VARHERE / LATEST` when created and restores all four when invoked (confirmed directly against forth6809.asm's `MARKERW / DOMARKER`), so running a section file and then letting it invoke its own marker leaves the dictionary, value-space and search chain exactly as they were right after `ttester.fs + 00_test_prelude.fs` were loaded — no leftover state for the next section file to trip over, whatever order they run in. (`BASE` is not part of that snapshot, see *Editing Test Files.*)
- 18_memory.tests.fs 's `MOVE` test still depends on the `FBUF / SBUF / SEEBUF` state left behind by the immediately preceding `FILL` test — this is fine, since both are in the same file and thus inside the same `MARKER` bracket.
- Several 20_numeric_output.tests.fs tests use `MAX-BASE` , `#BITS-UD` , and `s=` , all provided by `00_test_prelude.fs` , which sits outside every section's own `MARKER` bracket and is loaded once, up front.

The fixed load order is now simply: `00a_tool_ext_conditionals.fs` , `ttester.fs` , `00_test_prelude.fs` , then any subset of section files, in any sequence — see `ans_test_runner.py` (one level up), which automates

exactly this against either MAME or a real serial-connected MECB6809.

Test Reporting and Runner Changes

- **TEST-BEGIN / TEST-END** replace the old approach, where the runner sent its own reporting commands after each section and `ttester.fs` carried a `TEST-REPORT` word. Each section file now carries its own report, so the result is identical with or without the runner.
- **Pass/fail is decided by the TEST SUMMARY count**, computed in Forth from the real `T{ ... }T` comparisons. The earlier transcript scan for `ttester` error text could not tell a real test failure from the runner giving up on a missing `ok`, and is now only a secondary consistency warning.
- **Stack leak detection.** `TEST-END` reports `STACK IMBALANCE` and drops stray cells. Before this, leaks were invisible to `T{ ... }T`, and absolute `DEPTH` tests (sections 14 and 24) were only reliable if every earlier file had left the stack empty.
- **Compile-state tracking.** The runner's colon-definition tracking now scans raw tokens, so a `T{ : name ... ; -> }T` test is recognised as opening a definition and the runner no longer waits for an `ok` that a definition body line never sends.
- **ACCEPT tests.** The test line that follows `ACCEPT-TEST` is consumed by the target as input and is sent without waiting for an `ok`.
- **Flow control and diagnostics.** `XON/XOFF` is honoured, and `--trace-depth` probes `DEPTH` after each line.
- **Dead floating-point text** in `ttester.fs` is removed before sending, instead of being skipped by the target line by line.

Corrections to Test Content

- **Transcription bug, since fixed:** several "See F.x.x.xxxx WORD." cross-reference notes (and one "The following tests..." narrative line, in `08_defining_words.tests.fs`) had lost their leading `\` comment marker when the original web page's text was extracted — each would have been fed to the interpreter as literal, undefined words ("See", "F.6.1.0450", "The", "following", ...) the moment that file loaded. Found and fixed across all 17 section files while wiring up `ans_test_runner.py`.
- **Tests swallowed by comments.** In several files (08, 12, 19, 23, 24) tests had been glued to the end of a `\` comment line, so they never ran. They are now on their own lines.
- **TRUE and FALSE**. These were not dictionary words when the suite was

first organised. Both are now real words, and their tests
(F.6.2.1485 FALSE , F.6.2.2298 TRUE) are at the end of
26_abort_quit_headers.tests.fs .

- **BASE handling.** Sections 08, 11, 12, 24 and 26 change `BASE` and restore it explicitly. A bare `DECIMAL` in one file broke the next file in load order (and in section 24, broke the same file's own later `>NUMBER` tests).
- **16-bit cell adaptation (section 24).** The `>IN` test used `123456` , which wraps in a 16-bit cell. It uses `12345` , as the standard itself later did for 16-bit systems. The second `>IN` test had also lost the words it deliberately skips (`GCD calculation`); they are restored.
- **ENVIRONMENT? erratum (section 24).** The published `X:deferred` test,
`DUP 0= XOR INVERT -> <TRUE>` , can never pass on any system. It is replaced by a check that the answer is a single well-formed flag.
- **SM/REM (section 15).** One expected remainder broke the sign-of-dividend pattern of its neighbours and was corrected.
- **CATCH / THROW (section 11).** The undefined-word case expects `-13` .
- **Line layout.** Several original lines carried many tests each. They are one test per line now, apart from the layout-sensitive cases described in *Editing Test Files*.

Implementation Defects Found by the Suite

The tests exposed these defects in `forth6809.asm` . All are fixed, and all 17 sections now pass with a clean stack on MAME and via minicom.

- `2@ / 2!` had the cell order reversed (x2 must be at `addr` , x1 at `addr+2`).
`2CONSTANT` was corrected to match.
- **CATCH / THROW** did not save and restore the input source (`SRCADDR` , `SRCLLEN` , `SRCID` , `>IN`). A throw out of a nested `EVALUATE` lost the rest of the calling line and left stray stack cells. The frame now carries all four.
- **EVALUATE** now saves and restores the input source on the return stack.
- **ABORT** did not throw. It is now `-1 THROW` , and the top level prints nothing for `-1` .
- **UNLOOP and EXIT** . `UNLOOP` was a no-op and `EXIT` discarded the wrong number of loop-frame bytes (8 instead of 6). `UNLOOP` now drops the three-cell loop frame, and `EXIT` does not touch loop frames, as the standard requires (use `UNLOOP` before `EXIT`).

Known Limitations and Open Items

- **Compliance is not claimed.** Annex F is a necessary check, not proof of compliance. Many Annex F tests are adapted for a 16-bit cell and the suite excludes the word sets listed below.
- **Double-Number tests are not loaded.** The system implements part of the Double-Number word set (see *Tests Not Included*) but no Annex F double-number test file exists in `ans_tests` yet. Adding one, and implementing the missing words it needs, is the main open item.
- **ENVIRONMENT? coverage.** `X:deferred` is not in the query table, so it answers false. The test accepts either answer. `WORDLISTS` is deliberately absent (no Search-Order word set).
- **ABORT is silent.** It prints nothing and returns to the prompt, and `-1 THROW` at the top level also prints nothing.
`ABORT` message behaviour at the top level is not covered by the suite.
- **Layout-sensitive tests** (see *Editing Test Files*) cannot be reflowed, and pasting by hand needs the same care as the runner.
- **Floating-point build option.** `ttester.fs` stays in its original form with the floating branches in place. Only the runner removes them while sending. A manual paste of `ttester.fs` therefore runs slowly (every dead line passes through the target's `[ELSE]` skip).

Tests Not Included

The following tests are not included because the word set is not implemented or is implemented only in part:

1. Double-Number. *Partly implemented, no tests yet.*

Present: `D+` , `D-` , `DNEGATE` , `DABS` , `DMAX` , `DMIN` , `M+` , `S>D` , `D>S` ,
`D=` , `D<` , `DU<` , `D.` , `D.R` , `2ROT` , `2VARIABLE` , `2CONSTANT` ,
and double-number literals (`123.`).

Appear to be missing: `D0=` , `D0<` , `D2*` , `D2/` , `M*/` , `2LITERAL` , `2VALUE` .

Several of the present words are exercised indirectly by sections 15 and 20 (`M*` , `S>D` , `UM/MOD` , `>NUMBER` , `HOLD`), but there is no Annex F Double-Number test file.

2. Facility. Only `KEY?` is implemented. No tests are included for it.
3. File-Access.
4. Floating-Point.
5. Memory-Allocation.
6. Search-Order.

Search-Order's absence is documented explicitly
in the ClaudeForth documentation, Section 4.5.

Most of the Programming-Tools word set is also omitted
(AHEAD, CS-PICK, CS-ROLL, N>R, [THEN] etc.).
They have no ANS test coverage relevant here.

The Tools word set — `.S` , `WORDS` , `DUMP` are non-standard extensions,
so they have no official ANS test cases.